

JPA & Hibernate

The Java Persistence API, Hibernate as its engine, and how Spring Data JPA connects them to your Spring Boot application.

JPA

Hibernate

Spring Data

Entities

Relationships

Transactions

Table of Contents

01 JPA, Hibernate & Spring Data

Who does what in the stack

02 Connecting to Spring Boot

Dependencies and datasource configuration

03 Mapping Entities

@Entity, @Id, @Column and @MappedSuperclass

04 Relationships

One-to-many, many-to-one and fetch types

05 Repositories

JpaRepository, derived queries and @Query

06 Transactions & Persistence

@Transactional, lazy loading, the N+1 problem

07 Concurrency & Locking

Optimistic @Version and pessimistic locks

08 Cheat Sheet

Annotations and query recipes

1. JPA, Hibernate & Spring Data

These three names are often used interchangeably, but they are distinct layers. Understanding who does what removes most of the confusion.

Layer	What it is	Role
JPA	A <i>specification</i> (a set of interfaces & annotations)	Defines <i>how</i> to map objects to tables — but ships no code that runs.
Hibernate	The most popular <i>implementation</i> of JPA	The actual engine that generates SQL and talks to the database.
Spring Data JPA	A Spring layer <i>on top of</i> JPA/Hibernate	Generates repository implementations so you write almost no query code.

NOTE

Analogy: **JPA** is the interface, **Hibernate** is the class that implements it, and **Spring Data JPA** is the convenient wrapper that hands you ready-made repositories. You code against JPA annotations; Hibernate does the work; Spring Data removes the boilerplate.

What is ORM, and why?

Object-Relational Mapping bridges two worlds: Java *objects* (graphs of references) and relational *rows* (tables with foreign keys). Instead of writing SQL and copying `ResultSet` columns into fields by hand, you annotate a class and let Hibernate translate both directions.

```
// You write this...                                Hibernate runs this:
organizerRepository.save(organizer); // INSERT INTO organizer (name, email, ...) VALUES (.
organizerRepository.findById(42L);    // SELECT * FROM organizer WHERE id = 42
```

2. Connecting to Spring Boot

Add the starter and a driver

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa</artifactId> <!-- JPA + Hibernate -->
</dependency>
<dependency>
  <groupId>org.postgresql</groupId>
  <artifactId>postgresql</artifactId> <!-- JDBC driver -->
  <scope>runtime</scope>
</dependency>
```

Configure the datasource

Auto-configuration reads these properties and builds the `DataSource`, `EntityManagerFactory` and transaction manager for you:

```
spring:
  datasource:
    url: jdbc:postgresql://localhost:5432/eventticketing
    username: app
    password: secret
  jpa:
    hibernate:
      ddl-auto: validate      # validate schema against entities (see warning)
    show-sql: true           # log generated SQL while learning
    properties:
      hibernate.format_sql: true
```

ddl-auto value	Effect
none	Do nothing (use a migration tool — see below).
validate	Check that tables match entities; fail fast if not.
update	Alter the schema to match entities (handy in dev).
create-drop	Rebuild the schema on start, drop on stop (tests).

WATCH OUT

Never use `update` or `create-drop` in production — it can silently change or lose data. Manage schema with versioned migrations. The event-ticketing project uses **Flyway**: SQL files under `db/migration` run in order, and Hibernate is set to `validate`.

3. Mapping Entities

An **entity** is a class mapped to a table. Each instance is a row; each mapped field is a column. Hibernate tracks entities and syncs their changes to the database.

A base class for common columns

The project puts the id, optimistic-lock version and audit timestamps in a `@MappedSuperclass` so every entity inherits them (OOP inheritance at work):

```
@Getter @Setter
@MappedSuperclass           // not a table itself; contributes columns to subclasses
public abstract class BaseEntity {

    @Id                      // primary key
    @GeneratedValue(strategy = GenerationType.IDENTITY) // DB auto-increments it
    private Long id;

    @Version                  // optimistic locking (Section 7)
    @Column(name = "version", nullable = false)
    private Long version;

    @CreationTimestamp        // set once, on insert
    @Column(name = "created_at", nullable = false, updatable = false)
    private Instant createdAt;

    @UpdateTimestamp          // refreshed on every update
    @Column(name = "updated_at", nullable = false)
    private Instant updatedAt;
}
```

A concrete entity

```
@Entity // maps this class to a table
@Table(name = "organizer") // table name (defaults to the class name)
@Getter @Setter
@NoArgsConstructor // JPA REQUIRES a no-arg constructor
public class Organizer extends BaseEntity {

    @Column(name = "name", nullable = false)
    private String name;

    @Column(name = "email")
    private String email;

    @Column(name = "phone")
    private String phone;
}
```

Annotation	Purpose
<code>@Entity</code>	Marks the class as persistent.
<code>@Table(name=..)</code>	Overrides the default table name.
<code>@Id</code>	Marks the primary-key field.
<code>@GeneratedValue</code>	How the key is produced (<code>IDENTITY</code> , <code>SEQUENCE</code> , ...).
<code>@Column</code>	Column name, nullability, length, uniqueness.
<code>@Enumerated(EnumType.STRING)</code>	Store an enum as its name, not its ordinal.
<code>@Transient</code>	A field that must <i>not</i> be persisted.

WATCH OUT

Every entity needs a **no-argument constructor** (here supplied by Lombok's `@NoArgsConstructor`) — Hibernate uses it to instantiate rows via reflection.

4. Relationships

Foreign keys become object references. JPA maps them with four annotations; the fetch type and the *owning side* are the details that matter.

Annotation	Meaning	Default fetch
<code>@ManyToOne</code>	Many rows point to one (e.g. many events → one hall)	EAGER
<code>@OneToMany</code>	One row owns many (one hall → many seats)	LAZY
<code>@OneToOne</code>	One-to-one	EAGER
<code>@ManyToMany</code>	Join-table relationship	LAZY

```
@Entity
public class Event extends BaseEntity {

    @ManyToOne(fetch = FetchType.LAZY) // the FK lives on this side (owning)
    @JoinColumn(name = "hall_id", nullable = false)
    private Hall hall;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "organizer_id", nullable = false)
    private Organizer organizer;
}

@Entity
public class Hall extends BaseEntity {

    // inverse side: "mappedBy" names the field that owns the FK
    @OneToMany(mappedBy = "hall", cascade = CascadeType.ALL, orphanRemoval = true)
    private List<Seat> seats = new ArrayList<>();
}
```

Owning side, `mappedBy` and cascade

- The **owning side** holds the foreign-key column (`@JoinColumn`). It's the side Hibernate looks at when writing.
- The inverse side uses `mappedBy` to say “the other field owns this”.
- `cascade = ALL` propagates operations (save/delete) from parent to children; `orphanRemoval = true` deletes children removed from the collection.

TIP

Default to `FetchType.LAZY` for associations — even the `@ManyToOne` ones, which are eager by default. Eager fetching quietly loads half your database on every query. Load what you need explicitly with a fetch join (next section).

5. Repositories

This is where Spring Data JPA shines. Declare an *interface* and Spring generates the implementation at startup — CRUD, paging and custom queries included.

Extend JpaRepository and you get CRUD for free

```
public interface OrganizerRepository extends JpaRepository<Organizer, Long> {  
    // entity type ----- ^^^^^^^^^^  ^^^^ id type  
}
```

That empty interface already provides:

```
save(entity)      findById(id)      findAll()      findAll(Pageable)  
saveAll(entities) existsById(id)    count()       deleteById(id)
```

Derived queries — Spring parses the method name

```
public interface EventRepository extends JpaRepository<Event, Long> {  
  
    // SELECT * FROM event WHERE status IN (...) ORDER BY start_at ASC  
    List<Event> findByStatusInOrderByStartAtAsc(Collection<EventStatus> statuses);  
  
    Optional<Event> findByTitle(String title);  
    boolean existsByOrganizerId(Long organizerId);  
    long countByStatus(EventStatus status);  
}
```

Keyword in method name	SQL fragment
findBy, existsBy, countBy, deleteBy	SELECT / EXISTS / COUNT / DELETE
And, Or	AND / OR
In, Between, LessThan, Like	IN / BETWEEN / < / LIKE
OrderBy...Asc/Desc	ORDER BY

@Query for anything more complex (JPQL)

JPQL looks like SQL but is written against *entities and fields*, not tables and columns. Here is the project's locking query:

```
@Lock(LockModeType.PESSIMISTIC_WRITE)  
@Query("select e from Event e where e.id = :id")  
Optional<Event> findByIdForUpdate(@Param("id") Long id);
```

Use a **fetch join** to load an association in the same query and avoid the N+1 problem (Section 6):

```
@Query("select e from Event e join fetch e.hall where e.status = :status")
List<Event> findOnSaleWithHall(@Param("status") EventStatus status);
```

```
// Need real SQL? Set nativeQuery = true.
@Query(value = "SELECT * FROM event WHERE start_at > now()", nativeQuery = true)
List<Event> findUpcomingNative();
```

Paging and sorting

```
Page<Event> page = eventRepository.findAll(
    PageRequest.of(0, 20, Sort.by("startAt").descending()));

page.getContent();           // the 20 rows
page.getTotalElements();    // total across all pages
```

6. Transactions & the Persistence Context

Every database interaction happens inside a **transaction**, and Hibernate manages a **persistence context** (a.k.a. the first-level cache) that tracks your entities for the duration of that transaction.

@Transactional

```
@Service
public class OrganizerService {

    @Transactional // opens a tx, commits on success, rolls back on Runtime
    public OrganizerResponse create(CreateOrganizerRequest request) {
        Organizer o = new Organizer();
        o.setName(request.name());
        return OrganizerResponse.from(organizerRepository.save(o));
    }

    @Transactional(readOnly = true) // hint: no flushing, lighter for pure reads
    public List<OrganizerResponse> list() {
        return organizerRepository.findAll().stream()
            .map(OrganizerResponse::from).toList();
    }
}
```

Dirty checking — you often don't call save()

Inside a transaction, a loaded entity is *managed*. Change a field and Hibernate detects it (“dirty checking”) and issues an **UPDATE** at commit — no explicit save needed:

```
@Transactional
public void rename(Long id, String newName) {
    Organizer o = organizerRepository.findById(id).orElseThrow();
    o.setName(newName); // managed entity...
} // ...UPDATE runs automatically at commit
```

Lazy loading & the N+1 problem

```
// BAD: 1 query for events, then 1 query PER event to load its hall = N+1 queries
List<Event> events = eventRepository.findAll();
for (Event e : events) {
    System.out.println(e.getHall().getName()); // triggers a lazy SELECT each time
}
```

```
// GOOD: one query with a fetch join
List<Event> events = eventRepository.findOnSaleWithHall(EventStatus.ON_SALE);
```

WATCH OUT

LazyInitializationException happens when you touch a lazy association *after* the transaction (and its persistence context) has closed — typically in the controller/JSON layer. Fix it by fetching what you need inside the service and mapping to a DTO there, exactly as the project does.

TIP

Keep transactions on the **service layer**, not the controller or repository. The service is where a unit of business work (and its rollback boundary) belongs.

7. Concurrency & Locking

When two requests touch the same row at once, you need a strategy to avoid lost updates. JPA offers two.

Optimistic locking with @Version

Add a `@Version` column (the project's `BaseEntity` has one). Hibernate increments it on every update and checks it in the `WHERE` clause. If two transactions collide, the second fails with `OptimisticLockException` instead of silently overwriting:

```
UPDATE organizer SET name = ?, version = 6 WHERE id = ? AND version = 5;
-- 0 rows updated -> someone else changed it first -> OptimisticLockException
```

Best for low-contention data: no locks are held, so throughput stays high; you just retry on the rare conflict.

Pessimistic locking

For high-contention hot rows — like checking capacity so an event can't be oversold — take a real database lock up front. That is what `findByIdForUpdate` does (`SELECT ... FOR UPDATE`): concurrent bookings queue instead of racing.

```
@Lock(LockModeType.PESSIMISTIC_WRITE)
@Query("select e from Event e where e.id = :id")
Optional<Event> findByIdForUpdate(@Param("id") Long id);
```

	Optimistic (<code>@Version</code>)	Pessimistic (<code>FOR UPDATE</code>)
Locks the row?	No — detects conflict at commit	Yes — blocks other writers
Best for	Low contention	Hot, high-contention rows
On conflict	Exception → retry	Others wait

8. Cheat Sheet

THE STACK

- **JPA** = spec (annotations). **Hibernate** = engine (SQL). **Spring Data JPA** = auto-generated repositories.

MAPPING

<code>@Entity</code> / <code>@Table</code>	Class → table
<code>@Id</code> / <code>@GeneratedValue</code>	Primary key
<code>@Column</code> / <code>@Enumerated</code>	Field → column
<code>@ManyToOne</code> / <code>@OneToMany(mappedBy)</code>	Relationships
<code>@Version</code>	Optimistic locking

REPOSITORIES

- Extend `JpaRepository<Entity, Id>` for free CRUD + paging.
- Derived queries: `findByStatusOrderByStartAtAsc(...)`.
- Custom: `@Query` (JPQL, or `nativeQuery = true`); use `join fetch` to beat N+1.

TRANSACTIONS

- `@Transactional` on services; `readOnly = true` for pure reads.
- Managed entities are dirty-checked — a setter is enough to trigger an `UPDATE`.
- Map to DTOs inside the transaction to avoid `LazyInitializationException`.

PRODUCTION SCHEMA

- Set `ddl-auto: validate` and manage schema with **Flyway** migrations.

TIP

You now have the full picture: **Tutorial 01** (Java/OOP) → **Tutorial 02** (Spring Boot & REST) → **Tutorial 03** (JPA persistence). Together they are the exact stack the event-ticketing service is built on.